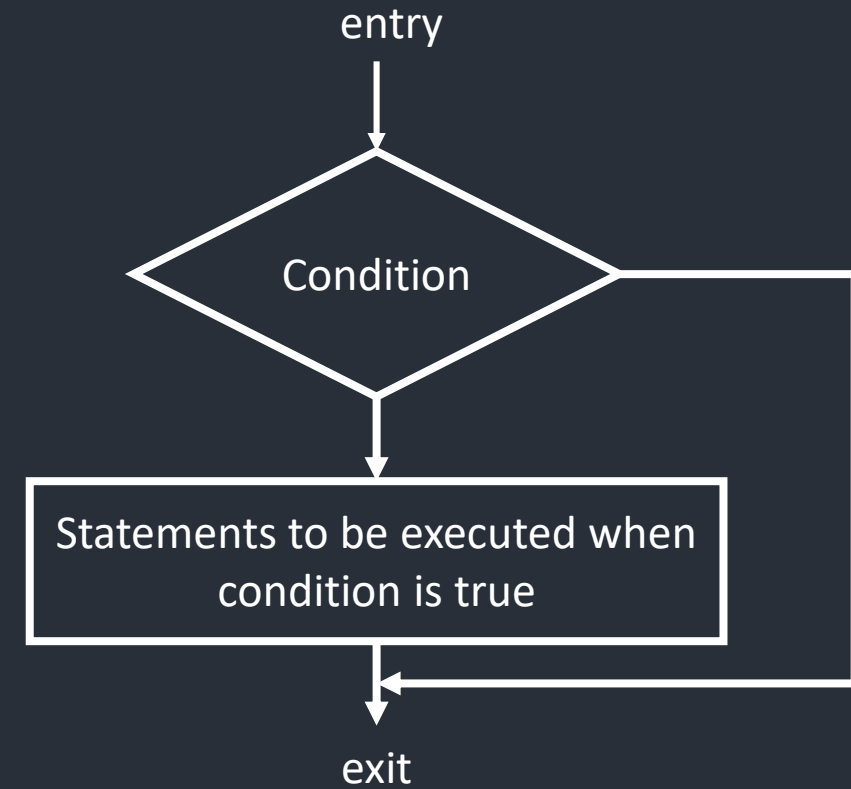




If Statement

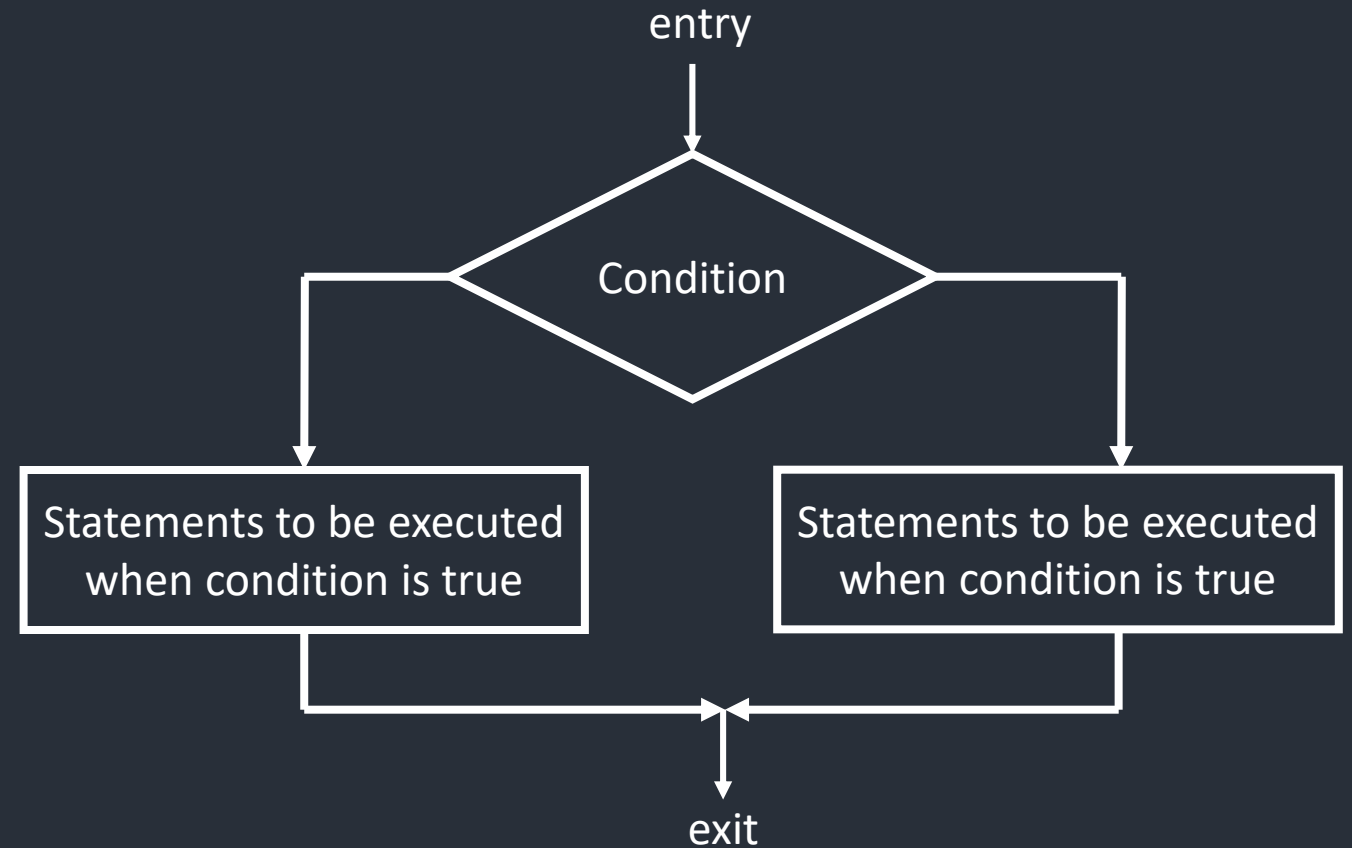
```
If (condition):  
    statements to be executed when  
    condition is true
```





If Else Statement

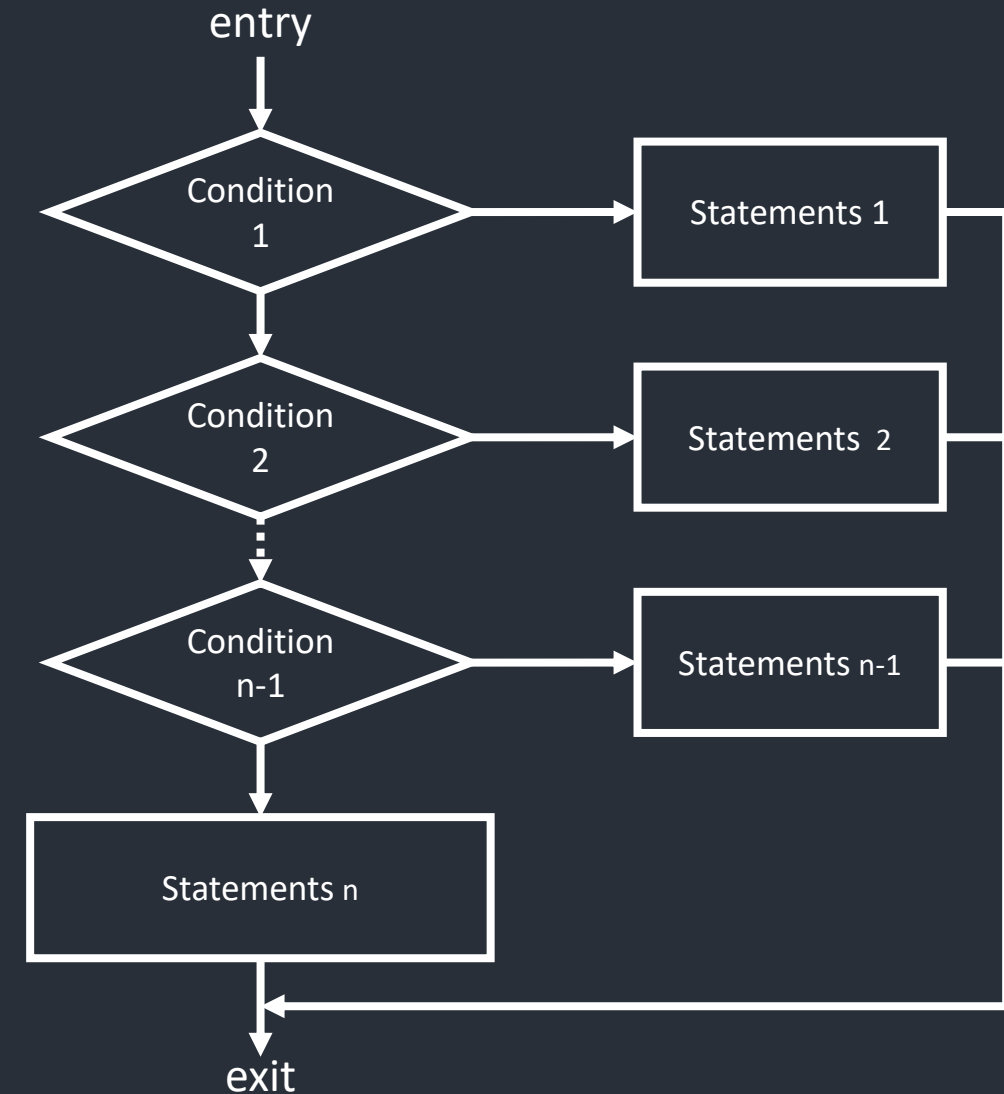
```
if (condition):  
    statements to be executed  
    when condition is true  
  
else:  
    statements to be executed  
    when condition is false
```





If Elif Else Statement

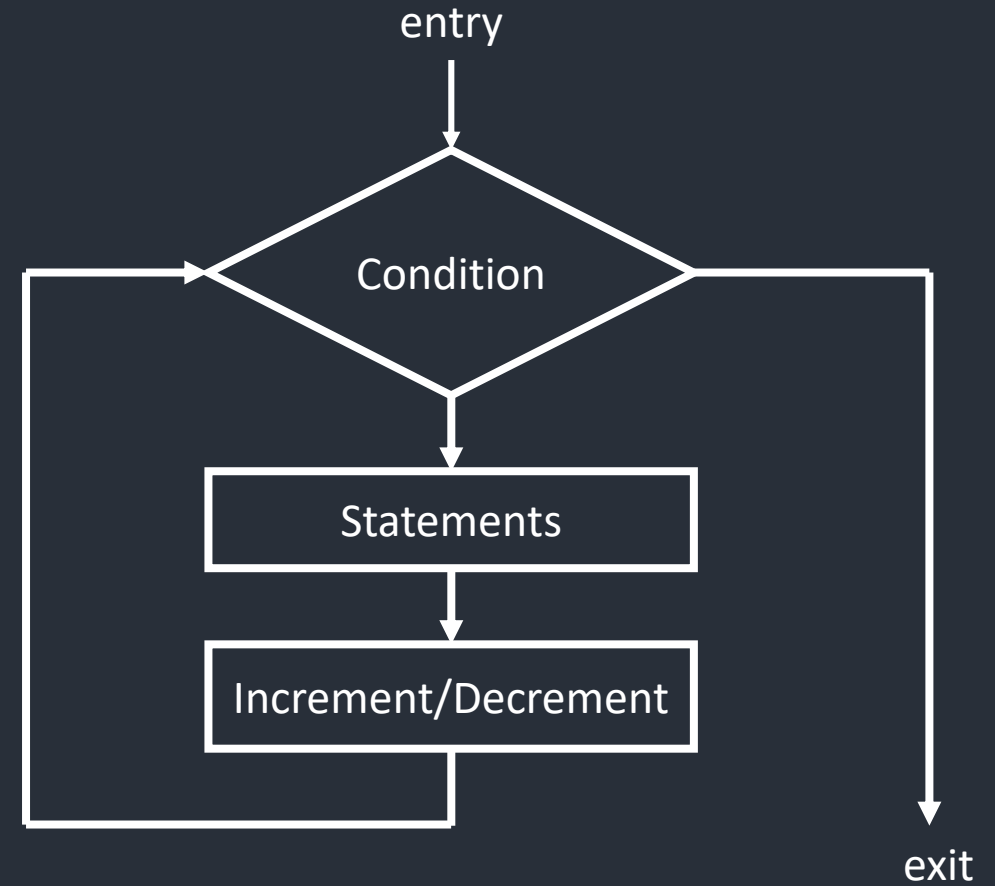
```
if (condition 1):  
    statements 1  
  
elif (condition 2):  
    statements 2  
  
elif (condition n-1):  
    statements n-1  
  
else:  
    statements n
```





While Loop

```
while (condition):  
    statements  
    increment/decrement
```





Range Function

The `range()` function in Python is used to generate a sequence of numbers, which can be used in loops or other iterable operations. It is commonly used in for loops for iteration.

Syntax:

```
range(start, stop, step)
```

Start : The starting value of the sequence (inclusive). Defaults to 0 if not specified. (optional)

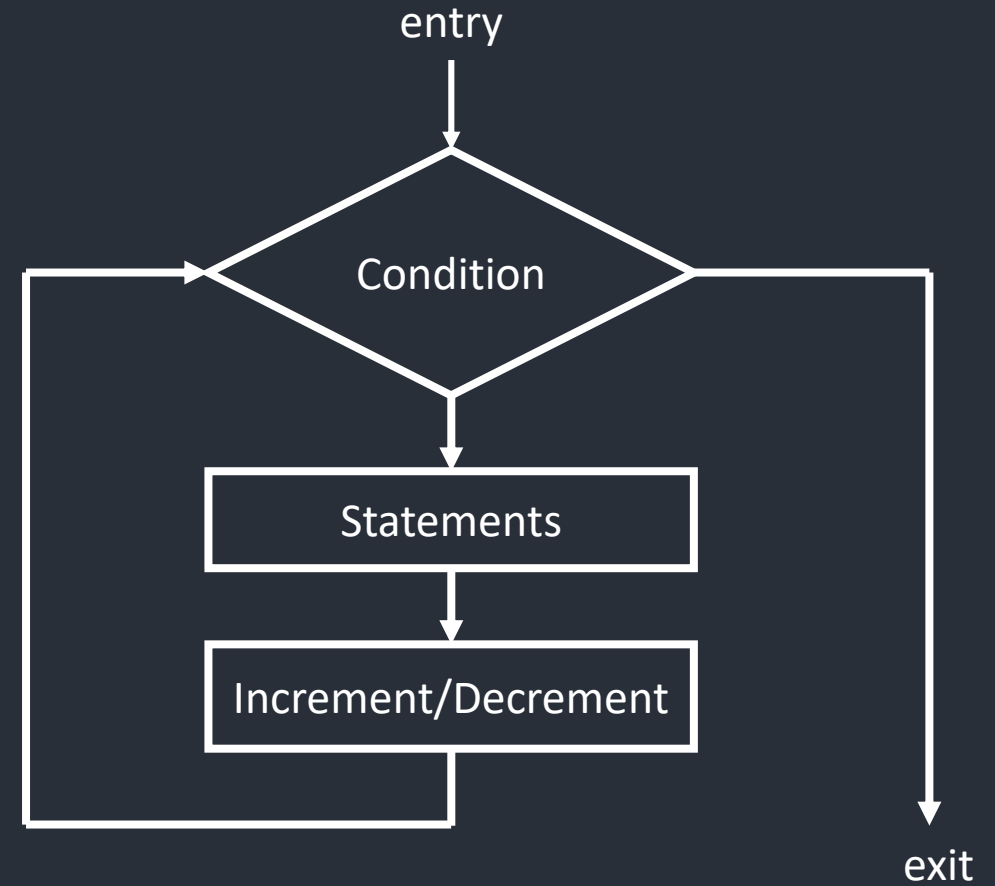
Stop : The ending value of the sequence (exclusive). (required)

Step : The increment (or decrement) between each number in the sequence. Defaults to 1. (optional):



For Loop

```
for in range (start, stop, step):  
    statements
```





Break & Continue Statements

The break and continue statements are used within loops in Python to alter the flow of execution. They are particularly useful for controlling iterations when specific conditions are met.

Break Statement:

- The break statement is used to exit a loop prematurely.
- When encountered, the loop terminates immediately, and control is passed to the first statement after the loop.

Continue Statement:

- The continue statement is used to skip the rest of the code in the current iteration and proceed to the next iteration of the loop.



Function

A function in Python is a block of reusable code that performs a specific task. Functions help organize code, making it more modular, readable, and maintainable. Functions can accept parameters (inputs) and return a value (output).



- The process of creating a function using the `def` keyword.
- Executing a function to perform its task by using its name followed by parentheses.
- Variables listed in the function's definition that accept input values when the function is called.
- The actual values passed to the function during a call.
- The `return` keyword is used to send a result back to the caller. A function without a return statement returns `None`.



Defining Function

In Python, functions are defined using the `def` keyword.

Syntax

```
def function_name(parameters):  
    """Docstring (optional)"""  
    # Function body  
    # Code to execute  
    return result # Optional
```

Example

```
def greet():  
    print("Hello, World!")  
  
def add(a, b):  
    sum = a + b  
    return sum
```

Key Components

- `def` — keyword to declare a function
- `function_name` — descriptive name following naming rules
- `parameters` — inputs the function accepts (optional)
- Function body — indented block of code
- `return` — sends a value back to the caller (optional)
- Without `return`, function returns `None`



Calling Function

A function is executed (called/invoked) by writing its name followed by parentheses. If the function expects parameters, you pass values (arguments) inside the parentheses.

Calling a Simple Function

```
# Define the function
def greet():
    print("Hello, World!")

# Call the function
greet()           # Output: Hello, World!
greet()           # Can be called multiple
times
```

Storing Return Values

```
def add(a, b):
    return a + b

result = add(3, 4)  # result = 7
print(result)
```

Calling with Arguments

```
def greet(name):
    print(f"Hello, {name}!")

greet("Thapathali")  # Hello, Thapathali!
greet("ECAST")       # Hello, ECAST!
```

How It Works

- Python jumps to the function body
- Executes the code inside the function
- Returns to the caller when done
- return value is captured by the caller



Arguments

Arguments are the actual values passed to a function when it is called. Parameters are the variables listed in the function definition. Python supports positional and keyword arguments.

Positional Arguments

Values are assigned to parameters based on their position (order). The number of arguments must match the number of parameters.

```
def describe(name, age, city):  
    print(f"{name} is {age}, from {city}")  
  
describe("Alice", 25, "New York")  
# Alice is 25, from New York  
  
describe(25, "Alice", "New York")  
# 25 is Alice, from New York (wrong!)
```

Parameter vs Argument

Parameter = variable in function definition | Argument = actual value passed during function call

Keyword Arguments

Values are assigned by explicitly naming each parameter. Order doesn't matter when using keyword arguments.

```
def describe(name, age, city):  
    print(f"{name} is {age}, from {city}")  
  
# Order doesn't matter with keywords  
describe(age=25, city="New York", name="Alice")  
# Alice is 25, from New York  
  
# Mixing: positional first, then keyword  
describe("Alice", city="New York", age=25)
```



Default Arguments

Default arguments allow you to provide a fallback value for a parameter. If the caller doesn't supply a value for that parameter, the default is used automatically. This makes functions more flexible.

Basic Default Parameters

```
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet("ECAST")    # Hello, ECAST!
greet()           # Hello, Guest! (default
                  # used)
```

Important Rules

- Default parameters must come AFTER non-default ones
- `def func(a, b=10)` is valid
- `def func(a=10, b)` is INVALID — `SyntaxError`
- Default values are evaluated only once at definition time
- Callers can always override default values

Multiple Default Parameters

```
def create_profile(name, age=18,
                  city="Unknown"):
    print(f"{name}, {age}, {city}")

create_profile("Alice")
# Alice, 18, Unknown

create_profile("Bob", 25)
# Bob, 25, Unknown

create_profile("Eve", 30, "LA")
# Eve, 30, LA
```



*args

*args allows a function to accept any number of positional arguments. Inside the function, args is a tuple containing all the extra positional arguments passed by the caller. This is useful when you don't know in advance how many arguments will be provided.

Syntax & Basic Example

```
def print_numbers(*args):  
    print(args)           # It's a tuple!  
    print(type(args))     # <class 'tuple'>  
    for num in args:  
        print(num)
```

```
print_numbers(1, 2, 3)  
# (1, 2, 3)  
# 1  
# 2  
# 3
```

Practical Example — Sum

```
def calculate_sum(*args):  
    total = 0  
    for num in args:  
        total += num  
    return total
```

```
print(calculate_sum(1, 2, 3))           # 6  
print(calculate_sum(10, 20, 30, 40))   # 100  
print(calculate_sum(5))                 # 5  
print(calculate_sum())                   # 0
```

Key Points

- *args collects extra positional arguments into a tuple
- The name args is a convention — *numbers also works
- Can be combined with regular parameters



*kwargs

****kwargs** allows a function to accept any number of keyword arguments. Inside the function, **kwargs** is a dictionary containing all the extra keyword arguments. This is useful for functions that need flexible, named parameters.

Syntax & Basic Example

```
def greet_person(**kwargs):
    print(kwargs)          # It's a dict!
    print(type(kwargs))    # <class 'dict'>
    for key, value in kwargs.items():
        print(f"{key}: {value}")

greet_person(name="Alice", age=30, city="NY")
# {'name': 'Alice', 'age': 30, 'city': 'NY'}
# name: Alice
# age: 30
# city: NY
```

Key Points

- ****kwargs** collects extra keyword args into a dictionary
- The name **kwargs** is a convention — ****options** also works
- Arguments must be passed as **key=value**

Practical Example — User Profile

```
def create_user(**kwargs):
    user = {}
    for key, val in kwargs.items():
        user[key] = val
    return user

profile = create_user(
    name="Bob",
    email="bob@mail.com",
    role="admin"
)
print(profile)
```

Combining All Parameter Types

Order: regular → *args → **kwargs



Lambda Function

A lambda function is a small, anonymous (unnamed) function defined with the lambda keyword. It can take any number of arguments but can only have one expression. Lambda functions are useful for short, throwaway operations.

Syntax

```
# Syntax:  
lambda arguments: expression
```

```
# Equivalent to:  
def function_name(arguments):  
    return expression
```

Basic Examples

```
# Add two numbers  
add = lambda x, y: x + y  
print(add(3, 5))    # 8
```

```
# Square a number  
square = lambda x: x ** 2  
print(square(4))    # 16
```

```
# Check even/odd  
is_even = lambda x: x % 2 == 0  
print(is_even(4))    # True
```



Pass Keyword

The pass keyword is a placeholder used to define a function (or class) without a body. Useful while planning code structure.

```
# Without pass – SyntaxError!  
def greet():
```

```
# With pass – works fine  
def greet():  
    pass
```

When to Use pass

- Placeholder while planning function implementations
- Empty class definitions
- In if/else blocks you want to handle later
- Prevents SyntaxError in empty code blocks